

Verifica dinamica

Marco Alberti



**Università
degli Studi
di Ferrara**

Ingegneria del Software Avanzata
CdLM in Ingegneria Informatica e dell'Automazione
A.A. 2021-2022

Ultima modifica: 5 giugno 2020

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright.
Ne sono vietati la riproduzione e il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore.

Sommario

- 1 L'attività di testing
- 2 Verifica in piccolo
 - Verifica white box
 - Verifica black box
- 3 Verifica in grande

Sommario

1 L'attività di testing

2 Verifica in piccolo

- Verifica white box
- Verifica black box

3 Verifica in grande

Test (o verifica dinamica)

Consiste nell'osservare il comportamento del sistema e confrontarlo con quello atteso.

Limitazioni:

- Non può (in generale) essere esaustivo: scelta, in base a caratteristiche del programma o del problema, di un insieme finito di casi di test
- I test possono dimostrare la presenza di errori, non la loro assenza (Dijkstra).
- Il software è purtroppo un manufatto con mancanza di continuità: un programma che funziona correttamente in un caso può fallire anche in un caso leggermente diverso

L'attività di test dovrebbe, idealmente,

- essere sistematica e integrata con altre tecniche (analisi statica).
- aiutare a localizzare gli errori, non solo rilevarli, per facilitarne la correzione (debugging)
- essere ripetibile
 - Influenza dell'ambiente di esecuzione (ad esempio, variabili non inizializzate)
 - Non-determinismo (concorrenza)
- essere oggettiva (utilità di specifiche formali)

Correttezza

- Un programma P che opera su input da un insieme D e produce output in un insieme R si può vedere come una funzione $P : D \rightarrow R$
- Dato un dato d del dominio D , $d \in D$, $P(d) \in R$ è il risultato dell'esecuzione di P con d come input

Correttezza:

- Sia $OR \subseteq D \times R$ la relazione input-output del programma (definita formalmente o informalmente)
- Dato $d \in D$, P è **corretto per** d se $(d, P(d)) \in OR$
- P è **corretto** se è corretto per ogni $d \in D$

Non-correttezza (terminologia IEEE)

- **Fallimento** (failure): P produce output non corretto per un certo d
- **Difetto** (defect): qualsiasi causa che può portare a un fallimento. Esempi:
 - errore di battitura
 - variabile non inizializzata
- **Malfunzionamento** (fault): stato intermedio scorretto che può essere assunto dal programma.
Esempio:
 - variabile che assume un valore inatteso
- **Errore**: l'azione umana che ha portato a un difetto
- Fallimento $\Rightarrow$ Malfunzionamento $\Rightarrow$ Difetto
- In generale non valgono le implicazioni inverse

Definizioni

- **Caso di test** per $P : D \rightarrow R$: è un elemento t di D
- **Insieme di test** (test set) T : insieme di casi di test, sottoinsieme di D
- P è **corretto per** T (ha successo per T) se è corretto per ogni suo elemento
- Dato $P : D \rightarrow R$, un test set T è **ideale** per P se la presenza di un difetto in P implica che P fallisca per qualche elemento di T .
- Un test set ideale permetterebbe la dimostrazione di correttezza di un programma tramite test (se T è ideale P è corretto per T , allora P è corretto)
- Nota: per un programma corretto, ogni test set è ideale.
- Fasi tipiche dell'esecuzione di un caso di test sul codice da verificare:
 - 1 *Arrange*: preparare l'input (e eventuali condizioni operative necessarie)
 - 2 *Act*: eseguire il codice
 - 3 *Assert*: valutare se l'output ha la proprietà desiderata

Criteri di selezione di test

- Dato $P : D \rightarrow R$, un **criterio di selezione** $C \subseteq 2^D$ è un insieme di test set per P
- Può essere definito come predicato sui test set.
- Se $T \in C$, T **soddisfa** C .
- Esempio:
 - se D è l'insieme degli interi a 32 bit, C può essere l'insieme dei test set che contengono almeno un negativo, 0 e un positivo.
 - $\{-4, 0, 5\} \in C$
 - $\{-4, 5, 7\} \notin C$

Coerenza e completezza

Coerenza:

- Un criterio C per un programma P è **coerente** se e solo se, per ogni $T_1, T_2 \in C$, P è corretto per T_1 se e solo se P è corretto per T_2 .
- Se un criterio è coerente, ogni suo elemento fornisce la stessa informazione sulla correttezza di P (stesso successo, ma non necessariamente provoca gli stessi fallimenti).

Completezza:

- Un criterio C per un programma P è **completo** se e solo se, in caso di scorrettezza di P , P non ha successo per almeno un insieme di test T selezionato da C .

Tutti i test set che soddisfano un criterio coerente e completo sono ideali: ogni test set permette di decidere la correttezza di P .

Esempio: coerenza e completezza

- $C_1 = \{T \mid T \subseteq \{0, 2\}\}$
- $C_2 = \{T \mid T \subseteq \{0, 1, 2, 3, 4\}\}$
- $C_3 = \{T \mid (\exists e \in T \mid e \geq 3)\}$

```
int main() {  
    int x, y;  
    scanf("%d", &x);  
    y = x * x;  
    printf("%d\n", y);  
    return 0;  
}
```

- C_1 coerente, ma non completo (sempre successo)
- C_2 completo, ma non coerente ($\{0, 4\}$ e $\{0, 2\}$ danno risultati opposti)
- C_3 coerente e completo

```
int main() {  
    int x, y;  
    scanf("%d", &x);  
    y = 5 + x;  
    printf("%d\n", y);  
    return 0;  
}
```

- C_1, C_2 diventano coerenti e completi
- C_3 è completo, ma non coerente (il test set $\{5\}$ non rileva malfunzionamenti).

Più fine di

Fissato un programma P , un criterio C_1 è **più fine** di un criterio C_2 se e solo se per ogni T_1 che soddisfa C_1 , esiste un $T_2 \subseteq T_1$ che soddisfa C_2

Esempio

Se D è l'insieme degli interi a 32 bit, siano

$C_1 = \{T \mid \exists \{n_1, 0, n_2\} \subseteq T \mid n_1 < 0, n_2 > 0\}$ (seleziona i test set che contengono un negativo, 0 e un positivo)

$C_2 = \{T \mid \exists \{n_1, n_2, n_3\} \subseteq T\}$ (seleziona i test set con almeno tre elementi)

- C_1 è più fine di C_2 (ogni insieme con un negativo, 0 e un positivo ha almeno tre elementi, quindi ha un sottoinsieme con tre elementi)
- C_2 non è più fine di C_1 (ad esempio, un insieme di cinque numeri positivi non ha sottoinsiemi contenenti 0)

Definizioni non verificabili

Non esistono algoritmi per verificare le definizioni date finora. In particolare:

- non esiste un algoritmo generale per decidere la correttezza di un programma
- non esiste un algoritmo generale per costruire un criterio completo e coerente che non sia l'eshaustività

Nella pratica, si utilizzano dei criteri di selezione che, in base alle caratteristiche del codice da testare, ci si aspetta selezionino test set

- significativi, cioè con un'alta probabilità di rilevare eventuali difetti
- di costo accettabile (se $T_1 \subset T_2$, T_2 è più significativo, ma costa di più)

La significatività non è funzione crescente della cardinalità: in

if ($x > y$) max = x; else max = x;

$\{(3, 2), (2, 3)\}$ rileva il difetto; $\{(3, 2), (4, 3), (5, 1)\}$ no.

Completa copertura

Un criterio empirico che tende a selezionare test set significativi:

Principio di completa copertura

Se è possibile esprimere il dominio D di input come $D = D_1 \cup D_2 \cup \dots \cup D_n$, dove gli elementi di ogni D_i causano un comportamento simile, un test set significativo si ottiene prendendo un rappresentante per ogni D_i .

Il criterio corrispondente è l'appartenenza al test set di un elemento rappresentativo di ogni D_i .

Ovviamente, la correttezza su un rappresentante di ogni D_i non implica la correttezza del programma.

Esempio

- Specifica del programma per il calcolo del fattoriale di un numero n :
 - Se $n < 0$, errore
 - Se $0 \leq n < 20$, restituire $n!$
 - Se $20 \leq n \leq 200$, approssimazione di $n!$ a meno dello 0.1%, ottenuta con calcoli in virgola mobile
 - Se $n > 200$, l'input può essere rifiutato
- $D = \{n \mid n < 0\} \cup \{0..19\} \cup \{20..200\} \cup \{n \mid n > 200\}$
- Un criterio è quello che individua i test set $\{n_1, n_2, n_3, n_4\}$ con $n_1 < 0$, $0 \leq n_2 \leq 19$, $20 \leq n_3 \leq 200$, $200 < n_4$.

Principio di completa copertura

- Se gli insiemi D_i non sono disgiunti, si possono scegliere rappresentanti nelle intersezioni per ridurre il numero di test. Esempio: se
$$D = \{d \mid d \bmod 2 = 0\} \cup \{d \mid d \leq 0\} \cup \{d \mid d \bmod 2 \neq 0\}$$
si ottiene completa copertura con $\{-37, 48\}$.
- Interpretazioni degeneri:
 - Un solo D_i
 - D_i composti da un solo elemento: esaustivo!
- Approssimazione di coerenza e completezza:
 - Si ottiene un criterio coerente suddividendo D in modo che, se due test set T_1 e T_2 differiscono solo per gli elementi rappresentativi di ogni D_i che li compongono, i risultati del test sono equivalenti (per quanto riguarda fallimento e successo)
 - Un criterio è completo se, per ogni errore possibile, esiste almeno un D_i tale che ogni suo elemento rileva l'errore.
- Non è possibile identificare automaticamente una scomposizione del dominio che garantisca coerenza e completezza

Sommario

1 L'attività di testing

2 Verifica in piccolo

- Verifica white box
- Verifica black box

3 Verifica in grande

Sommario

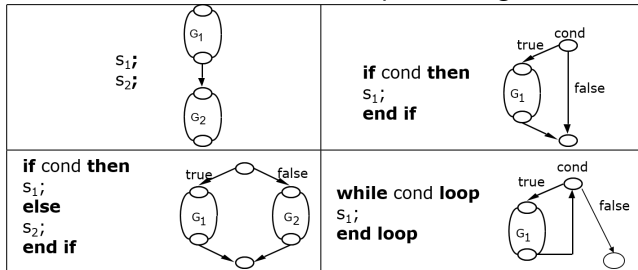
- 1 L'attività di testing
- 2 Verifica in piccolo
 - Verifica white box
 - Verifica black box
- 3 Verifica in grande

Criteri strutturali di selezione dei test (approccio white box)

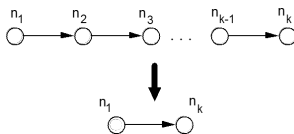
- Applicano il principio di completa copertura il dominio suddiviso in base alle parti del programma eseguite dato ogni input.
- Ad esempio:
 - se il programma è composto da n istruzioni, la suddivisione $D = D_1 \cup \dots \cup D_n$ è tale che gli input in D_i causano l'esecuzione dell'istruzione i -esima.
 - Selezionando un input per ogni D_i si ottiene completa copertura.
 - Ovviamente i D_i non sono necessariamente disgiunti.
- Utilizzano la struttura interna del programma per identificare i casi di test:
 - Copertura delle istruzioni
 - Copertura delle decisioni (o degli archi)
 - Copertura delle condizioni
 - Copertura dei cammini

Grafo di controllo

- Rappresentazione del programma, costruita ricorsivamente
- Ogni istruzione elementare è rappresentata da un nodo
- Siano s_1 e s_2 due istruzioni, e G_1 e G_2 i corrispondenti grafi. Allora:

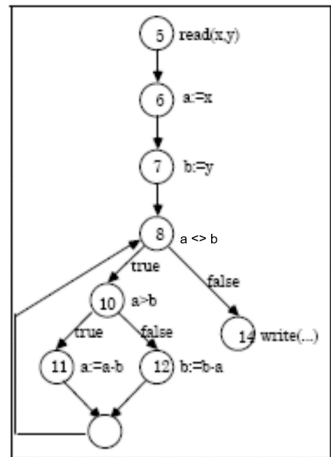


- Una sequenza di archi si può collassare in un solo arco:



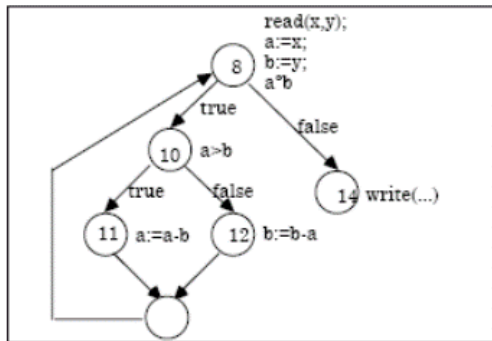
Esempio: algoritmo di Euclide

```
1  program gcd (input, output);  
2  var  
3    x, y, a, b: integer;  
4  begin  
5    read (x, y);  
6    a := x;  
7    b := y;  
8    while a <> b do  
9      begin  
10       if a > b  
11         then a := a - b;  
12         else b := b - a  
13       end;  
14    write ('MDC dei dati e'', a)  
15  end.
```

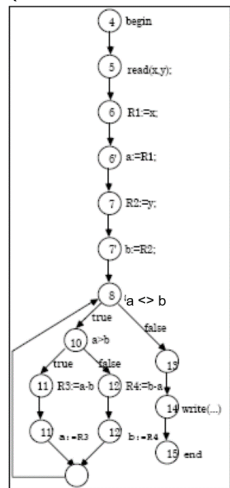


Esempio: algoritmo di Euclide

Sequenza collassata:



Espansione dell'assegnamento
(R1, ..., R4 registri):



Criterio di copertura delle istruzioni

- Un test set T rispetta **criterio di copertura delle istruzioni** se e solo se, eseguendo il programma P con ogni elemento di T , ciascuna istruzione elementare di P è eseguita almeno una volta.
- Istruzione elementare: ad esempio, nei linguaggi imperativi, assegnamenti, operazioni di I/O e chiamate di subroutine/metodi.
- Nel grafo di controllo, ogni nodo deve essere percorso almeno per un caso di test.
- Misura di copertura $C_0 = \frac{\text{n. istruzioni eseguite}}{\text{n. istruzioni elementari}}$

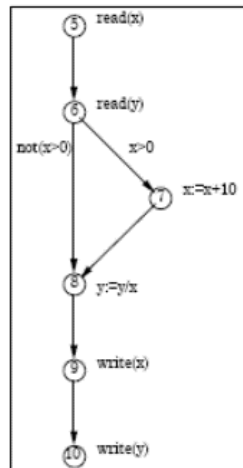
Esempio (algoritmo di Euclide)

```
begin
  read(x); read(y);
  while x != y loop
    if x > y then
      x:= x - y;
    else
      y:= y - x;
    end if;
  end loop;
  gcd := x;
end;
```

Si ottiene copertura delle istruzioni ($C_0 = 1$) con il test set $\{(3, 3), (4, 3), (3, 4)\}$

Esempio

```
1 Program statement (input, output);  
2   var  
3     x,y : real;  
4   begin  
5     read(x);  
6     read(y);  
7     if x > 0 then x:=x+10;  
8     y:=y/x;  
9     write(x);  
10    write(y);  
11  end.
```



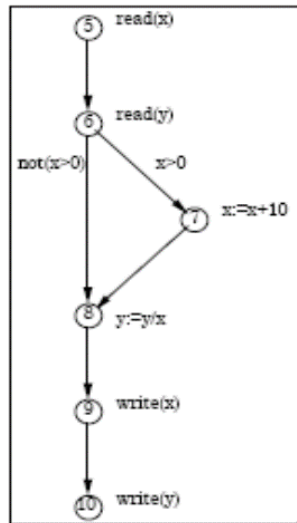
Un test con $x > 0$ e y qualsiasi (ad esempio $\{(20, 30)\}$) garantisce la copertura delle istruzioni, ma non rileva l'errore che si ha per $x = 0$ alla riga 8.

2. Copertura degli archi (o delle decisioni)

- Un test set T rispetta il **criterio di copertura degli archi** se e solo se, eseguendo il programma per ogni elemento di T , ogni arco del grafo di controllo del programma è percorso almeno una volta.
- Più fine del criterio di copertura delle istruzioni (identifica test set più grandi)
- Misura di copertura $C_1 = \frac{\text{n. archi percorsi}}{\text{n. archi percorribili}}$

Esempio

- Il test set $\{(20, 30)\}$ visto prima non garantisce la copertura degli archi.
- Il test set $\{(20, 30), (0, 30)\}$ garantisce la copertura degli archi e rileva l'errore nella riga 8 (divisione per 0).
- Il criterio di copertura degli archi è più fine del criterio di copertura delle istruzioni



Debolezza del criterio: esempio

```
found := false; counter := 1;
while (not found) and counter < number_of_items loop
  if table (counter) = desired_element then
    found := true;
  end if;
  counter := counter + 1;
end loop;
if found then
  write ("the element is in the table");
else
  write ("the desired element is not in the table");
end if;
```

Il test set: (1) tabella vuota, (2) tabella con 3 elementi, di cui il secondo è l'elemento ricercato copre gli archi, ma non rileva l'errore ($<$ invece di $\leq$).

3. Copertura delle condizioni

- Generalmente, le condizioni logiche sono composte da formule atomiche (espressioni relazionali o variabili booleane) e loro negazioni, composte tramite congiunzioni e disgiunzioni
- Ogni formula atomica o sua negazione è detta **costituente** della condizione.
- Un test set T rispetta il **criterio di opertura delle condizioni** se e solo se, eseguendo il programma per ogni elemento di T , tutti gli archi del grafo di controllo risultano percorsi, e tutti i possibili valori dei costituenti delle condizioni composte risultano esercitati almeno una volta.

Debolezza del criterio

```
if x <> 0 then
  y := 0;
else
  z := z - x;
end if;
if z > 1 then
  z := z / y;
else
  z := 0;
end if;
```

$\{(x = 0, z = 3), (x = 1, z = 1)\}$ copre gli archi e le condizioni, ma non rileva la possibilità di una divisione per 0 (ad es. per $x = 1, z = 3$)

4. Copertura dei cammini

- Un test set T rispetta il **criterio di copertura dei cammini** se e solo se l'esecuzione del programma per ogni elemento di T provoca l'esecuzione di tutti i cammini che collegano il nodo iniziale a quello finale nel grafo di controllo.
- Più fine del criterio di copertura degli archi

Esempio

Il test set $\{(0, 1), (0, 2), (1, 1), (1, 2)\}$ copre i cammini (e rileva la divisione per 0).

- Il numero di cammini può essere elevato (o infinito in caso di cicli) anche per programmi semplici.
- In presenza di istruzioni condizionali (if o case) occorre cercare di combinare il più possibile i cammini (e in maniera oculata)

Ricerca in tabella

```
found := false; counter := 1;
while (not found) and counter < number_of_items loop
  if table (counter) = desired_element then
    found := true;
    end if;
    counter := counter + 1;
end loop;
if found then
  write ("elemento presente");
else
  write ("elemento non presente");
end if;
```

Ci sono infiniti cammini; in questo caso, è impossibile coprirli tutti anche con un test set infinito (il ciclo termina dopo al massimo `number_of_items - 1` iterazioni)

Copertura in presenza di cicli

Linee guida in presenza di cicli: scegliere il test set in modo che il ciclo venga eseguito nelle seguenti modalità:

- Il numero minimo di volte compatibile con la struttura di controllo
- Il massimo numero di volte (se esiste)
- Un numero di volte intermedio

Esempio

Nel caso della ricerca in tabella, almeno tre casi:

- tabella vuota;
- non vuota e elemento presente (meglio se articolato in posizione prima, ultima o intermedia)
- non vuota e elemento assente.

Individuazione dei test set

Una volta scelto un criterio, bisogna individuare i test set che lo soddisfano.

Esempio

```
read(x);  
z:=2*x;  
if z=4 then  
  statement
```

Per eseguire `statement` occorre dare 2 come input per x.

- In generale, ovviamente il test set dovrà avere cardinalità maggiore di 1.
- Salvo altre motivazioni, è bene cercare di minimizzare la cardinalità

Non fattibilità

In generale non è possibile garantire la copertura completa. Qui `statement` non è raggiungibile:

Esempio

```
if x>0 then
  if x<0 then
    statement
```

In generale non è possibile determinare alitmicamente la raggiungibilità delle istruzioni.

Sommario

1 L'attività di testing

2 **Verifica in piccolo**

- Verifica white box
- **Verifica black box**

3 Verifica in grande

White box vs Black box

Difetti test white box:

- Copertura legata all'implementazione: una modifica dell'implementazione richiede di rivalutare i casi di test.
- Non rilevano la mancata implementazione di funzionalità.
- Non applicabile al test-driven development, che richiede di scrivere i test prima del codice da testare

Alternativa:

- Test **black-box**, basati sulla specifica di un modulo, anziché sulla sua implementazione.
- I test set si individuano in base a casistiche ricavate dalla specifica (formale o informale).
- La generazione dei casi di test può essere automatizzata (property based testing)
- A seconda del tipo di specifica, varie nozioni di copertura (es. nelle specifiche logiche, copertura dei valori costituenti della preconditione)

Esempio: casi di test da specifica informale

Il programma riceve in ingresso una registrazione che descrive una fattura (viene fornita una descrizione dettagliata del formato della registrazione).

La fattura deve essere inserita in un file di fatture ordinato per data. La fattura deve essere inserita nella posizione corretta del file. Se esistono altre fatture che riportano la stessa data, la nuova fattura deve essere inserita dopo l'ultima già presente.

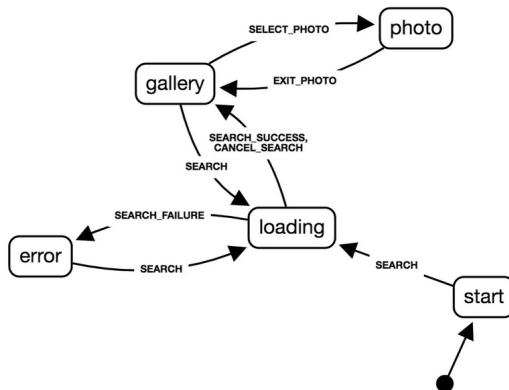
Inoltre devono essere eseguite alcune verifiche di coerenza: il programma dovrebbe verificare se il cliente è già presente nel corrispondente archivio dei clienti, se i dati del cliente nei due file corrispondono, etc.

Casi di test:

- Fattura con data seguente a quelle presenti
- Fattura con data uguale all'ultima presente
- Fattura con data precedente all'ultima presente (illegale?). Sottocasi:
 - Fattura con stessa data di una fattura già presente
 - Fattura con data diversa da tutte quelle già presenti
- Diverse fatture scorrette

Verifica da FSM

- Singolo caso di test:
 - 1 partenza nello stato iniziale
 - 2 applicazione di una sequenza di input
 - 3 verifica che lo stato sia quello previsto
- Criteri di copertura:
 - sequenze di input
 - stati
 - cammini
- Più l'implementazione della FSM è diretta, più facilmente i criteri di copertura si traducono in quelli corrispondenti sul codice



Verifica da specifiche logiche

Assezioni input/output, contratti:

$\{\text{Pre } (i1, i2, \dots, in) \} P \{\text{Post } (o1, o2, \dots, om, i1, i2, \dots, in)\}$

- Arrange: caso di test che rispetta la preconditione
- Act: esecuzione di P
- Assert: verifica della postcondizione

Esempio

Divisione intera:

$\{i1 > i2\}$

P

$\{i1 = i2 * o1 + o2 \text{ and } o2 \geq 0 \text{ and } o2 < i2\}$

Casi di test: tutti con $i1 > i2$, eventualmente valutare robustezza con $i2$ zero o negativo (specificazione insufficiente?)

Verifica da specifiche algebriche

410_verifica_dinamica/list.casl

```
1 spec LIST [sort Elem pred le: Elem * Elem] =
2   generated type
3     List ::= nil | cons(Elem;List)
4   ops
5     ins: Elem * List -> List
6   preds
7     sorted: List
8   vars e,h:Elem; l,t:List
9     . ins(e,nil) = cons(e,nil);
10    . le(e,h) => ins(e,cons(h,t)) = cons(e(cons(h,t)));
11    . not le(e,h) => ins(e,cons(h,t)) = cons(h,ins(e,t));
12    . sorted(nil);
13    . sorted(cons(e,nil));
14    . sorted(cons(e,cons(h,t))) <=> le(e,h) /\ sorted(cons(h,t))
15 end
```


Verifica black box: specifiche algebriche

- Proprietà da verificare: gli assiomi
- casi di test: generazione di valori per le variabili degli assiomi
 - casuali
 - applicazioni di sequenze di generatori

Esempio

Verifica dell'implementazione di `sorted` rispetto alle sue proprietà

12 : per input `nil`, atteso risultato `True`

13 : verifica (ev.te per vari valori di `e`)

14 : per ognuno di vari valori generati per `t` (`nil` e varie applicazioni di `cons`, mantenendo ordinamento o no) e per tutti gli ordinamenti di `e` e `h`, verificare che i due membri della doppia implicazione abbiano lo stesso valore logico

Sommario

1 L'attività di testing

2 Verifica in piccolo

- Verifica white box
- Verifica black box

3 Verifica in grande

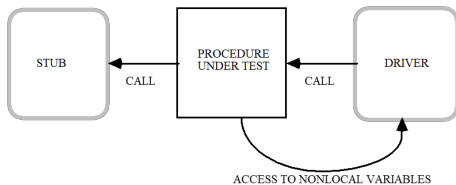
Test e modularità

- La modularità di un progetto guida l'attività di test.
- Attività:
 - Test di modulo (singolo)
 - Test di integrazione: interazioni fra moduli
 - Test di sistema (complessivo)
 - Test sul sistema con partecipazione degli utenti

Test di modulo

- Un modulo non può essere testato senza le funzionalità che importa.
- Se un modulo M_i richiede M_h e M_k (M_i **USES** M_h , M_i **USES** M_k), e questi non sono disponibili, è necessario simulare le loro risorse.
- Inoltre può essere necessario testare la chiamata di M_i da parte di un altro modulo.
- Scaffolding (ponteggio): ambiente per il test di modulo.

Test di un modulo funzionale



Stub:

- Modulo di interfaccia è uguale a uno non ancora sviluppato, e che per i casi di test dà gli stessi risultati (da tabella, file, operatore umano)
- Utilità di specifiche eseguibili (es. logiche) da cui derivare gli stub
- Variante: **Mock**, che oltre a dare gli stessi risultati all'interfaccia effettua le stesse chiamate ad altri moduli.

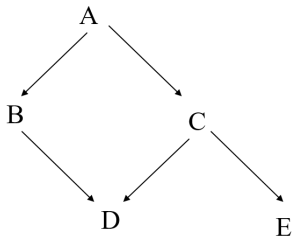
Driver:

- Programma che simula l'invocazione del modulo sotto test.
- Procede all'inizializzazione del modulo sotto esame e alla chiamata di operazioni significative e in sequenze significative.

Test di integrazione

Vantaggi di test di integrazione incrementali:

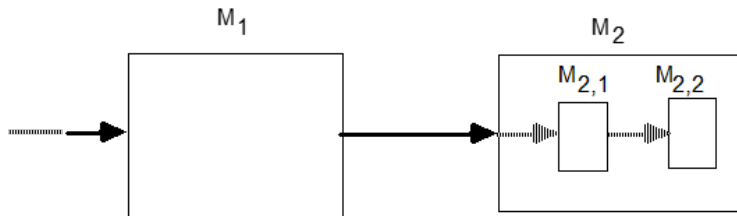
- si può anticipare il test di integrazione a quando sono pronti i moduli interessati
- è più facile localizzare gli errori
- è opportuno testare insieme moduli correlati
- un'aggregazione parziale di moduli può costituire una fornitura parziale (anche per test di accettazione)
- può ridurre la necessità di stub e driver



Se la relazione **USES** è una gerarchia:

- Bottom-up: inizio del test dalle foglie e sviluppo di driver
- Top-down: inizio del test dalla radice e sviluppo di stub.

Esempio: combinazione degli approcci



M_1 **USES** M_2 e M_2 **IS-COMPOSED-OF** $\{M_{2,1}, M_{2,2}\}$. Possibilità:

- Test di M_1 , con stub per M_2 e driver per M_1 . Poi implementazione di $M_{2,1}$ e stub per $M_{2,2}$.
- Implementazione di $M_{2,2}$ e test con driver. Implementazione di $M_{2,1}$ e test di M_2 con driver. Implementazione di M_1 e test con driver.

Test di software object-oriented: ereditarietà

- L'ereditarietà permette il riuso di codice mediante specializzazione.
- Anche i test, quando possibile, andrebbero riutilizzati.
- In linea di massima sarebbe opportuno:
 - non ripetere il test degli elementi (metodi) invariati rispetto alla superclasse
 - testare i nuovi metodi e quelli ridefiniti
- Possono però esserci interazioni.
- Considerare ogni classe come unità indipendente? Si perde l'incrementalità
- Annotare (manualmente) i casi di test:
 - Test che non va ripetuto per nessun erede
 - Test che va ripetuto per la classe erede X e tutti i suoi eredi
 - Test che va ripetuto con gli stessi input
 - Test che va ripetuto modificando gli input

Test di codice generico

- Se i test su `Table(Int)` hanno successo, cosa si può dire dei test su `Table(Invoice)`?
- Più il codice è generico, più è facile da testare
 - Es: ordinamento basato su operazione generica (`Precedes` anziché `<`)
- Spazio delle possibili istanziazioni dei tipi parametrici:
 - Se è noto, si può procedere esaustivamente
 - Altrimenti, suddivisione in classi (tipi primitivi, strutturati, etc.) e principio di completa copertura

Test di sistema

- Spesso coinvolge componenti non software (hardware, operatori)
- Si concentra sulle interazioni fra componenti
- Test di scenari completi di utilizzo (end-to-end): ad esempio, in applicazione a strati, azione che li coinvolga tutti
- Tecnica dello scaffolding non limitata al software (ad es. simulatori di dispositivi hardware)
- A seconda della struttura del sistema, la modifica di un componente può comportare o meno un nuovo test di sistema

Test con utenti

Hanno lo scopo di raccogliere le osservazioni degli utenti sul software sviluppato o in sviluppo, per apportare correzioni eventualmente necessarie.

- Alpha test: software ancora incompleto. Per valutare:
 - utilità
 - usabilità
 - eventuali funzionalità mancanti
- Beta test: software funzionalmente completo, ma non testato in tutte le condizioni operative. Valuta:
 - completezza della funzionalità
 - robustezza rispetto alle diverse condizioni operative (hardware, software)
- Test di accettazione: software completo. Gli utenti approvano il software per l'utilizzo previsto, o richiedono correttivi.

Altri tipi di test

- Non solo correttezza funzionale
- Test di tutte le qualità:
 - Prestazioni
 - Resistenza al sovraccarico
 - Sicurezza
 - Più in generale, test di robustezza
- Test di regressione
 - Le successive modifiche al software possono introdurre nuovi errori o manifestarne di già esistenti
 - Ripetere, dopo ogni modifica, i test non resi obsoleti.